

Where do you want to start?

You already have JDK 25 locally — just zip up the JDK install directory itself (e.g. Temurin 25 distribution), so you can drop it onto machines that don't have it. No special export steps.

25 distribution) so you can drop it onto machines that don't have it. No special export step needed, it's just files.

b) Maven `.m2` repo (the important one)

Don't just zip your existing `~/m2` — it'll have partial/corrupt entries. Do a clean pull instead:

```
bash
# From backend/ with pom.xml fully fleshed out (all deps declared)
mvn dependency:go-offline -Dmaven.repo.local=/path/to/clean-m2
```

This forces Maven to resolve and download **every** dependency (including plugins, their dependencies, and transitive deps) into a fresh local repo folder. Also explicitly run a full build once against that repo to catch anything `go-offline` misses (some plugins fetch things only during actual `package / test` phases):

```
bash
mvn clean package -Dmaven.repo.local=/path/to/clean-m2
```

Then zip `/path/to/clean-m2` — that's your transferable `.m2`.

Critical: also copy your `pom.xml`'s exact plugin versions (compiler plugin, surefire, spring-boot-maven-plugin, jasperreports-maven-plugin if you use it) — pin every version explicitly. Don't rely on version ranges or "latest" — offline, Maven can't resolve a moving target.

c) Node / npm side

Two viable approaches — pick one:

Option 1 (simplest, works well for a single project): ship `node_modules` directly

```
bash
cd frontend
npm ci # clean install from package-lock.json
```

Then zip `node_modules/` **built on the same OS** as the offline target (native modules like `esbuild`, `sharp`, `node-sass` compile OS-specific binaries — a `node_modules` built on Windows won't work on Linux and vice versa). This is fine for one project but doesn't scale if you'll add more JS projects later.

Option 2 (better long-term): local npm cache + offline registry

```
bash
npm ci
npm cache verify
```

Then zip your npm cache folder (find it via `npm config get cache`, typically `~/npm`). On the offline machine, point npm at this cache and run `npm ci --offline`. This is more portable across projects but a bit more setup.

For a single stable stack like yours, **Option 1 is simpler and less to go wrong** — go with that unless you expect to spin up more frontend projects later.

Also zip the **Node.js binary/installer itself** — the offline machine needs Node runtime, not just `node_modules`.

d) PostgreSQL

Download the full installer (Windows `.exe` / Linux `.deb` / `.rpm` or the binary tarball) for your target OS — not just the client. If you'll containerize, also pull the Docker image and `docker save` it to a `.tar` (only if Docker itself is allowed offline, which needs its own bundling — flag this if relevant, it's a common gotcha since Docker Desktop wants to phone home for updates).

e) IDE and misc tools

- IDE installer (IntelliJ/VSCode) + any offline plugin bundles (Lombok plugin, Spring plugin, etc. — these are also downloadable as offline plugin zips from the IDE's plugin marketplace).
- Jasper Studio if you're hand-designing `.jrxml` templates visually (bundle its installer too).
- Git itself, if not already on target machines.
- Any Chrome/browser dev tools extensions you rely on.

3. Transfer

Checksum everything before and after transfer (`sha256sum`) — corrupted zips of a 500MB `.m2` are the single most common cause of "works on staging, fails offline" support tickets. Write the checksums into `docs/offline-setup/versions.txt` alongside exact version numbers of every component.

4. On the offline machine — set up

1. **Install JDK** — unzip, set `JAVA_HOME`, add to `PATH`.
2. **Install Node** — unzip/install binary, confirm `node -v` / `npm -v`.
3. **Configure Maven to use local-only repo:** In `~/m2/settings.xml`:

```
xml
<settings>
  <mirrors>
    <mirror>
      <id>offline-repo</id>
      <mirrorOf>*</mirrorOf>
      <url>file:///path/to/clean-m2</url>
    </mirror>
  </mirrors>
  <offline>true</offline>
</settings>
```

Then just `mvn clean package` from `backend/` — it should resolve entirely from the local mirror with zero network calls.

4. **Frontend:** copy the pre-built `node_modules` next to `package.json`, then just run `npm run start /build` directly — skip `npm install` entirely since `node_modules` is already there (running `npm install` offline without a registry will just fail/hang, so don't trigger it by accident — e.g. don't delete `node_modules` before testing).
5. **PostgreSQL:** run the installer, create the DB, restore schema via your migration scripts.
6. **IDE:** install from the offline installer, then install plugins from the downloaded plugin zips via "Install Plugin from Disk" rather than the marketplace.
7. **Validate immediately**, in this order, before declaring the machine ready:
 - `mvn clean package` succeeds with zero network hits (temporarily disconnect network to be sure).
 - `npm run build` succeeds.
 - Generate one JasperReports PDF and one Excel export end-to-end.
 - Connect backend to PostgreSQL and run one migration.

That last validation step matters most — it's much cheaper to discover a JasperReports/JDK 25 mismatch or a missing transitive Maven dependency *now*, while you can still go fetch one missing jar, than after the machine is fully sealed off.

If you want, I can help draft the actual `pom.xml` dependency block (Spring Boot + JasperReports + PostgreSQL driver) with pinned versions, or a shell/PowerShell script that automates steps 2–4 on the staging machine — just let me know which OS your dev machines run.

🔍 🗨️ 📄 🔄



Write a message...



Sonnet 5 Medium ▾ 🗣️ 📄

Claude is AI and can make mistakes. Please double-check cited sources.